

IASI Dev User Guide

Herramientas de desarrollo y operación del ecosistema IASI

IASI Project

Table of contents

1	Introducción	4
1.1	Cómo se escribe una instrucción	4
1.2	Recorrido recomendado	4
2	Propósito del documento	6
2.1	Workspace	6
2.2	Git y GitHub	6
2.3	Repositorio	7
2.4	Destino	7
2.5	Publicación	7
2.6	Fuentes	7
2.7	Formato de salida	8
2.8	Construir y build	8
2.9	Resultado de construcción	8
2.10	Artefacto publicable	8
2.11	Publicar, publish y publish/	8
2.12	Cambio Git	9
2.13	Remoto y push	9
2.14	Confirmar, commit y commit	9
2.15	Desplegar y deploy	10
2.16	Despliegue completo y deploy --full	10
2.17	Flujo documental	10
2.18	Clonar	11
2.19	Actualizar con pull	11
2.20	Sincronizar con sync	11
2.21	Log	11
2.22	Elegir la operación	12
3	Requisitos	13
3.1	Workspace recomendado	13
3.2	Hacer accesible el comando	14
3.3	Inicialización completa	14
3.4	Logs	15

4	Ayuda	16
4.1	<code>clone</code>	16
4.2	<code>pull</code>	16
4.3	<code>sync</code>	17
4.4	<code>build</code>	17
4.5	<code>publish</code>	18
4.6	<code>commit</code>	18
4.7	<code>deploy</code>	18
4.8	<code>docker</code>	19
5	Trabajo diario en una publicación	20
5.1	Publicación completa de un repositorio	20
5.2	Despliegue completo	20
5.3	Actualizar el workspace desde GitHub	21
5.4	Incorporar un repositorio ausente	21
5.5	Propagar un recurso común	22
5.6	Gestionar servicios locales	22
6	Operaciones que pueden sobrescribir trabajo	23
6.1	El comando no se encuentra	23
6.2	Un subcomando rechaza una opción	23
6.3	No se encuentra ninguna publicación	24
6.4	Faltan <code>Rscript</code> o Quarto	24
6.5	Fallo de GitHub o Git	24
6.6	No hay cambios para desplegar	25
6.7	Fallo de Docker	25
6.8	Consultar el log correcto	25

1 Introducción

`iasi-dev` es una herramienta que se utiliza escribiendo instrucciones en una terminal. Esas instrucciones permiten realizar las tareas habituales de desarrollo y operación del ecosistema IASI.

Permite preparar un workspace, actualizar repositorios, construir y publicar documentación, desplegar cambios y gestionar los servicios locales con una interfaz común.

Esta guía está dirigida a las personas que trabajan con la documentación y los proyectos IASI. Explica el vocabulario utilizado, qué instrucción elegir, qué resultado esperar y qué precauciones tomar.

1.1 Cómo se escribe una instrucción

```
iasi-dev <comando> [opciones]
```

Para consultar la ayuda disponible:

```
iasi-dev help  
iasi-dev help <comando>
```

En esta guía, llamamos **comando** a la palabra que indica la operación principal, como `build` o `deploy`. Llamamos **opción** a un modificador precedido por `-` o `--`, como `--full`. Los valores entre corchetes son opcionales; los corchetes no se escriben.

1.2 Recorrido recomendado

- empiece por **Conceptos y nomenclatura**;
- prepare su entorno con **Primeros pasos**;
- consulte **Comandos** cuando necesite conocer una operación concreta;
- utilice **Workflows recomendados** para realizar tareas completas;
- acuda a **Seguridad y diagnóstico** ante una duda o un fallo.

El siguiente capítulo define las palabras utilizadas en el resto del manual. Ningún nombre de operación implica por sí solo las demás: construir, publicar, confirmar y desplegar son acciones diferentes.

2 Propósito del documento

Las herramientas utilizadas por IASI comparten palabras como *publicar* o *desplegar*, pero no siempre les atribuyen el mismo significado.

En este documento, cada término tiene una definición concreta. Cuando decimos **construir**, **publicar**, **confirmar** o **desplegar**, nos referimos exclusivamente a las operaciones descritas aquí.

2.1 Workspace

En este documento, cuando decimos **workspace**, nos referimos al directorio que contiene los repositorios IASI como carpetas hermanas.

```
iasi-org/  
  iasi-common/  
  iasi-quarto/  
  iasi-quarto-docs/  
  iasi-tools-dev/
```

El workspace permite ejecutar una misma operación sobre varios repositorios. No es en sí mismo un repositorio Git.

2.2 Git y GitHub

En este documento, cuando decimos **Git**, nos referimos al sistema que registra la historia y los cambios de los archivos de un proyecto.

En este documento, cuando decimos **GitHub**, nos referimos al servicio donde se alojan y comparten los repositorios remotos de IASI.

2.3 Repositorio

En este documento, cuando decimos **repositorio**, nos referimos a un directorio gestionado por Git y conectado normalmente con un repositorio remoto en GitHub.

Un workspace contiene varios repositorios. Una operación puede dirigirse a todo el workspace, a un repositorio o a un directorio más concreto.

2.4 Destino

En este documento, cuando decimos **destino**, nos referimos a la ruta indicada al final de un comando para limitar su alcance.

```
iasi-dev build iasi-quarto-docs/01-user-guide
```

En este ejemplo, `iasi-quarto-docs/01-user-guide` es el destino. Cuando se omite, muchos comandos utilizan el directorio actual.

Los comandos que seleccionan publicaciones o repositorios siguen una misma regla: sin destinos procesan todos los aplicables; con uno procesan solo ese destino; con varios procesan exactamente la selección indicada.

2.5 Publicación

En este documento, cuando decimos **publicación**, nos referimos a un proyecto documental reconocido por `iasi.quarto` que puede generar uno o varios formatos de salida.

Una publicación puede ser, por ejemplo, una guía de usuario. Un repositorio puede contener una sola publicación o varias.

2.6 Fuentes

En este documento, cuando decimos **fuentes**, nos referimos a los archivos editables a partir de los que se genera una publicación: documentos QMD, configuración, imágenes y otros recursos.

Las fuentes no son todavía el sitio HTML ni el PDF final.

2.7 Formato de salida

En este documento, cuando decimos **formato de salida**, nos referimos a una de las formas finales que puede generar una publicación, como HTML o PDF.

2.8 Construir y build

En este documento, cuando decimos **construir**, nos referimos a transformar las fuentes en uno o varios formatos de salida que puedan revisarse.

El comando correspondiente es:

```
iasi-dev build [destino]
```

Construir no crea commits, no realiza **push** y no despliega cambios. Tampoco significa necesariamente que el contenido de **publish/** haya sido actualizado.

2.9 Resultado de construcción

En este documento, cuando decimos **resultado de construcción**, nos referimos al HTML, PDF u otro formato generado por **build**.

Estos resultados sirven para comprobar la publicación antes de prepararla para el despliegue.

2.10 Artefacto publicable

En este documento, cuando decimos **artefacto publicable**, nos referimos a un archivo o directorio preparado para ser distribuido, como un sitio HTML o un PDF.

2.11 Publicar, publish y publish/

En este documento, cuando decimos **publicar** como operación, nos referimos a reunir resultados ya contruidos dentro del directorio **publish/**.

El comando correspondiente es:

```
iasi-dev publish [destino]
```

En este documento, cuando escribimos **publish/**, nos referimos al directorio que contiene los artefactos publicables preparados para su despliegue.

publish no sustituye a **build**: prepara resultados existentes. Tampoco crea commits ni realiza **push**.

2.12 Cambio Git

En este documento, cuando decimos **cambio Git**, nos referimos a una diferencia local que Git puede registrar: un archivo añadido, modificado o eliminado.

Un resultado construido puede producir cambios Git si sus archivos se guardan dentro del repositorio.

2.13 Remoto y push

En este documento, cuando decimos **remoto**, nos referimos al repositorio Git compartido, normalmente alojado en GitHub.

En este documento, cuando decimos **realizar push**, nos referimos a enviar al remoto los commits locales.

2.14 Confirmar, commit y commit

En este documento, cuando decimos **confirmar cambios**, nos referimos a crear un commit de Git que registra un conjunto de cambios con un mensaje.

El comando `iasi-dev commit` añade todos los cambios del repositorio seleccionado, crea el commit y lo envía al remoto:

```
iasi-dev commit -m "mensaje" [repositorio]
```

Por tanto, en esta guía **commit** no significa únicamente crear el commit local: el comando también realiza **push**.

2.15 Desplegar y deploy

En este documento, cuando decimos **desplegar**, nos referimos a confirmar los cambios de uno o varios repositorios y enviarlos al remoto.

El comando correspondiente es:

```
iasi-dev deploy [destino]
```

Cuando existe `publish/`, `deploy` separa los cambios ordinarios y los artefactos publicables en commits distintos antes de realizar `push`.

`deploy` sin opciones no construye ni publica. Utiliza el estado que ya existe.

2.16 Despliegue completo y deploy --full

En este documento, cuando decimos **despliegue completo**, nos referimos a ejecutar consecutivamente:

1. `build`;
2. `publish`;
3. `deploy`.

El comando correspondiente es:

```
iasi-dev deploy --full [destino]
```

Si la construcción o la publicación falla, el despliegue no continúa.

2.17 Flujo documental

Con las definiciones anteriores, el recorrido completo es:

```
fuentes
→ build
→ resultados de construcción
→ publish
→ artefactos en publish/
→ deploy
→ commits enviados al remoto
```

Durante la edición normal se repite **build** y se revisan los resultados. **publish** se utiliza cuando los resultados ya están preparados. **deploy** se reserva para enviar cambios.

2.18 Clonar

En este documento, cuando decimos **clonar**, nos referimos a obtener una nueva copia local de un repositorio remoto.

El comando `iasi-dev clone` trabaja con el conjunto de repositorios IASI y, por defecto, recrea las copias existentes. `clone --resume` conserva las que ya existen y añade solo las ausentes.

2.19 Actualizar con pull

En este documento, cuando decimos **actualizar con iasi-dev pull**, nos referimos a sustituir el estado local por el estado de la rama remota predeterminada.

No debe confundirse con una integración conservadora: el comando elimina cambios locales y archivos no seguidos.

`pull` opera siempre sobre repositorios completos. Para seleccionar uno o varios, recibe únicamente los nombres de sus directorios raíz, donde se encuentra `.git`. Sin nombres, selecciona todos los repositorios aplicables.

2.20 Sincronizar con sync

En este documento, cuando decimos **sincronizar**, nos referimos a copiar desde `iasi-common` un archivo o directorio compartido hacia las copias existentes en el workspace.

`sync` no consulta GitHub, no crea commits y no despliega los cambios copiados.

2.21 Log

En este documento, cuando decimos **log**, nos referimos al archivo con el registro detallado de una ejecución.

Los logs se guardan normalmente bajo `logs/` y utilizan una marca temporal para distinguir cada ejecución.

2.22 Elegir la operación

Quiero...	Operación
preparar por primera vez todo el entorno IASI	<code>init</code>
recuperar todos los repositorios desde cero	<code>clone</code>
añadir únicamente los repositorios que faltan	<code>clone --resume</code>
sustituir el estado local por el remoto	<code>pull</code>
distribuir un recurso de <code>iasi-common</code>	<code>sync</code>
generar resultados revisables	<code>build</code>
preparar artefactos en <code>publish/</code>	<code>publish</code>
confirmar y enviar el estado ya preparado	<code>deploy</code>
construir, publicar y desplegar	<code>deploy --full</code>
confirmar y enviar todos los cambios sin tratamiento documental	<code>commit</code>
gestionar los servicios locales	<code>docker</code>

3 Requisitos

`iasi-dev` se ejecuta con Bash. Además, cada comando necesita las herramientas relacionadas con su tarea:

Herramienta	Comandos que la necesitan
Git	<code>clone</code> , <code>pull</code> , <code>commit</code> , <code>deploy</code>
GitHub CLI (<code>gh</code>)	descubrimiento y clonación de repositorios de la organización
R y Rscript	<code>build</code> , <code>publish</code> e instalación de <code>iasi.quarto</code>
Quarto	construcción de publicaciones
Docker con Compose	<code>docker</code> e inicialización de servicios

Antes de comenzar, autentique GitHub CLI:

```
gh auth status
```

3.1 Workspace recomendado

Los repositorios IASI se mantienen como directorios hermanos dentro de un workspace común:

```
iasi-org/  
  iasi-common/  
  iasi-quarto/  
  iasi-quarto-docs/  
  iasi-tools-dev/  
  logs/
```

Esta disposición es importante para los comandos que operan sobre varios repositorios y para `sync`, que toma `iasi-common` como origen.

3.2 Hacer accesible el comando

Añada el directorio `bin` de `iasi-tools-dev` a `PATH`. Después compruebe:

```
iasi-dev help
```

También puede invocarlo mediante su ruta completa desde Bash:

```
/ruta/al/workspace/iasi-tools-dev/bin/iasi-dev help
```

3.3 Inicialización completa

Para preparar un workspace nuevo:

```
iasi-dev init /ruta/al/workspace
```

La inicialización ejecuta, por orden:

1. recreación de los repositorios;
2. instalación de `iasi.quarto`;
3. configuración de `iasi-lua` en los proyectos Quarto;
4. arranque de los contenedores Docker.

`init` incluye una operación destructiva de clonación. Revise el capítulo de seguridad antes de usarlo sobre un workspace que contenga trabajo local.

Opciones comunes de inicialización:

```
iasi-dev init -v /ruta/al/workspace # más detalle
iasi-dev init -s /ruta/al/workspace # modo silencioso
iasi-dev init -y /ruta/al/workspace # sin confirmación
```

Al terminar, compruebe el entorno con:

```
iasi-dev docker status
iasi-dev build /ruta/al/workspace/iasi-tools-dev-docs/01-user-guide
```

Si ya dispone de un workspace con cambios propios, no utilice `init` para actualizarlo. Elija `clone --resume` para completar los repositorios ausentes o actualice manualmente los repositorios cuyo trabajo necesite conservar.

3.4 Logs

La salida detallada de las herramientas subyacentes se guarda en archivos con marca temporal dentro de `logs/`. La consola muestra principalmente mensajes de estado de IASI.

Cuando una operación falle, conserve la ruta del log indicada por el comando: contiene el diagnóstico que normalmente no aparece completo en pantalla.

La opción `-v` muestra más mensajes útiles durante la ejecución, pero el log sigue siendo la fuente principal para investigar un fallo.

4 Ayuda

```
iasi-dev help
iasi-dev help build
```

La ayuda del propio CLI es la referencia más inmediata para las opciones disponibles en la versión instalada.

4.1 clone

```
iasi-dev clone [opciones] [workspace]
```

Obtiene de GitHub la lista de repositorios no archivados de la organización y los clona en el workspace.

Por defecto recrea los destinos: si un repositorio ya existe, se elimina antes de clonarlo. Use `--resume` para clonar únicamente los repositorios que todavía no existen.

```
iasi-dev clone --resume /ruta/al/workspace
```

Opciones: `-v`, `-s`, `-y/--yes` y `-r/--resume`.

4.2 pull

```
iasi-dev pull [opciones] [repositorio...]
```

`pull` utiliza el workspace `iasi-org`. Sin argumento sincroniza todos sus repositorios. Para actualizar uno o varios, se indican los nombres de sus directorios dentro de `iasi-org`.

En este comando no se utilizan rutas completas ni directorios internos del repositorio. El directorio indicado es siempre la raíz del repositorio y contiene `.git`.

`pull` prioriza deliberadamente el remoto: restablece el repositorio local a la rama remota predeterminada y elimina archivos no seguidos. No equivale a un `git pull` conservador.

```
cd /c/iasi-org
iasi-dev pull iasi-quarto-docs
iasi-dev pull iasi-quarto-docs iasi-lua-docs
```

Opciones: `-v`, `-s` y `-y/--yes`.

4.3 sync

```
iasi-dev sync ruta [ruta...]
```

Propaga desde `iasi-common` archivos o directorios que ya tengan copias con el mismo nombre en el workspace.

```
iasi-dev sync resources
iasi-dev sync _quarto.yml resources
```

Los directorios se reemplazan completos, incluidos los archivos eliminados en el origen. No se crean copias donde el nombre todavía no exista. Si hay más de una entrada con el mismo nombre en `iasi-common`, el comando se detiene para evitar elegir un origen ambiguo.

4.4 build

```
iasi-dev build [-v] [repositorio-o-directorio...]
```

Busca publicaciones IASI Quarto por debajo del destino y construye sus formatos configurados. Los directorios llamados `tests` se excluyen.

El destino puede ser una ruta anidada:

```
iasi-dev build iasi-quarto-docs/01-user-guide
```

En ese caso se procesa esa publicación y las publicaciones situadas por debajo de ella; la operación no se amplía automáticamente a todo el repositorio Git contenedor.

Sin argumento, la búsqueda comienza en el directorio actual:

```
cd /ruta/al/workspace/iasi-quarto-docs
iasi-dev build
```

4.5 publish

```
iasi-dev publish [-v] [repositorio-o-directorio...]
```

Ensambla en `publish/` los artefactos que ya existen. No sustituye la construcción previa. En un multiproyecto, las publicaciones se reúnen bajo el `publish/` de la raíz.

También admite destinos anidados con el mismo alcance que `build`.

Utilice `publish` cuando ya haya revisado la construcción y quiera preparar el contenido que posteriormente se desplegará.

4.6 commit

```
iasi-dev commit -m "mensaje" [repositorio...]
```

Añade todos los cambios, crea un commit y lo envía al remoto. Sin repositorio procesa los repositorios Git situados directamente bajo el directorio actual.

El mensaje es obligatorio. Revise siempre `git status` y el diff antes de ejecutar el comando, porque incluye todos los cambios del repositorio seleccionado.

`commit` no construye ni prepara publicaciones y no da un tratamiento separado a `publish/`. Para desplegar documentación resulta más adecuado `deploy`.

4.7 deploy

```
iasi-dev deploy [-f|--full] [-m "mensaje"] [repositorio...]
```

Despliega los cambios mediante commits y `push`. Cuando existe `publish/`, separa los cambios ordinarios y los artefactos publicados en commits distintos.

Sin `--full` no construye ni publica: despliega el estado que ya exista. Con `--full`, exige primero que `build` y `publish` terminen correctamente.

```
iasi-dev deploy --full -m "actualiza manual" ./iasi-quarto-docs
```

El mensaje predeterminado es **deploy**.

Cuando se indica un workspace, el comando procesa sus repositorios. Cuando se indica un repositorio, limita el despliegue a ese destino. Pase siempre una ruta explícita si no desea depender del directorio actual.

4.8 docker

```
iasi-dev docker [start|stop|status]
```

Gestiona los servicios declarados en **docker/iasi-compose.yml**. Sin subcomando utiliza **start**.

La configuración actual incluye PlantUML, Ollama y Open WebUI. Los puertos y volúmenes efectivos deben consultarse en el archivo Compose de la versión instalada.

5 Trabajo diario en una publicación

Para editar y comprobar una publicación concreta:

```
iasi-dev build iasi-quarto-docs/01-user-guide
```

Revise el resultado generado. Cuando esté listo para preparar los artefactos publicables:

```
iasi-dev publish iasi-quarto-docs/01-user-guide
```

El uso de una ruta anidada evita reconstruir todas las publicaciones del repositorio.

Este es el flujo recomendado mientras se escribe: repetir `build`, revisar y corregir. No es necesario ejecutar `publish` después de cada cambio.

5.1 Publicación completa de un repositorio

Desde el workspace:

```
iasi-dev build ./iasi-quarto-docs  
iasi-dev publish ./iasi-quarto-docs
```

Separe construcción y publicación cuando quiera inspeccionar los resultados antes de alterar `publish/`.

5.2 Despliegue completo

Cuando quiere construir, publicar y enviar los cambios en una sola operación:

```
iasi-dev deploy --full -m "actualiza documentación" ./iasi-quarto-docs
```

Antes de hacerlo:

1. revise el estado Git;
2. confirme que el destino es el repositorio correcto;
3. asegúrese de que no hay cambios ajenos que no deban incluirse;
4. compruebe la autenticación y el remoto configurado.

Si prefiere revisar cada fase, utilice el flujo explícito:

```
iasi-dev build ./iasi-quarto-docs
# revisar los resultados
iasi-dev publish ./iasi-quarto-docs
# revisar publish/
iasi-dev deploy -m "actualiza documentación" ./iasi-quarto-docs
```

deploy sin --full utiliza los archivos que ya están preparados y evita repetir la construcción.

5.3 Actualizar el workspace desde GitHub

Si no hay trabajo local que conservar:

```
cd /c/iasi-org
iasi-dev pull -y
```

Para un único repositorio:

```
iasi-dev pull -y iasi-quarto-docs
iasi-dev pull -y iasi-quarto-docs iasi-lua-docs
```

El argumento es el nombre del directorio situado directamente dentro de `iasi-org`. No se indica una ruta completa ni un subdirectorio como `iasi-quarto-docs/01-user-guide`.

No utilice este flujo para integrar cambios remotos con trabajo local: `pull` reemplaza el estado completo del repositorio local.

Si existe cualquier duda, ejecute primero `git status` dentro de los repositorios afectados y haga una copia externa.

5.4 Incorporar un repositorio ausente

Para completar un workspace sin recrear lo existente:

```
iasi-dev clone --resume /ruta/al/workspace
```

5.5 Propagar un recurso común

Después de revisar el origen en `iasi-common`:

```
iasi-dev sync nombre-del-recurso
```

Revise después los diffs de todos los repositorios modificados. `sync` copia contenido; no crea commits ni publica los cambios.

5.6 Gestionar servicios locales

```
iasi-dev docker start  
iasi-dev docker status  
iasi-dev docker stop
```

Use `status` antes de diagnosticar como fallo documental un servicio que simplemente no está iniciado.

6 Operaciones que pueden sobrescribir trabajo

Preste especial atención a estos comandos:

Comando	Riesgo principal
<code>clone</code>	elimina destinos existentes antes de clonarlos, salvo con <code>--resume</code>
<code>pull</code>	restablece al remoto y elimina archivos no seguidos
<code>sync</code> sobre un directorio <code>commit</code>	reemplaza completamente las copias existentes incluye todos los cambios del repositorio seleccionado
<code>deploy</code>	crea commits y realiza <code>push</code>

Antes de utilizarlos, revise `git status`, confirme las rutas resueltas y haga una copia externa de cualquier trabajo que no esté confirmado.

Las opciones `-y` y `--yes` eliminan la confirmación; no reducen el alcance ni hacen recuperable la operación.

6.1 El comando no se encuentra

Compruebe que está usando Bash y que el directorio `iasi-tools-dev/bin` pertenece a `PATH`:

```
command -v iasi-dev
iasi-dev help
```

6.2 Un subcomando rechaza una opción

Las opciones no son globales. Consulte la ayuda específica:

```
iasi-dev help <comando>
```

Por ejemplo, `-s` está disponible en algunas operaciones de `workspace`, pero no en `build` o `publish`.

6.3 No se encuentra ninguna publicación

Compruebe que el destino contiene una publicación reconocible por `iasi.quarto` y que no está dentro de un directorio `tests`, excluido del descubrimiento.

Si solo desea una publicación anidada, pase su ruta exacta. Si desea todo el multiproyecto, pase la raíz que contiene las publicaciones.

También puede haberse ejecutado el comando desde un directorio distinto del esperado. Repita la operación con una ruta explícita para eliminar esa ambigüedad.

6.4 Faltan Rscript o Quarto

Compruebe que ambos comandos están disponibles desde la misma sesión de Bash en la que ejecuta `iasi-dev`:

```
command -v Rscript
command -v quarto
```

La presencia del CLI `iasi-dev` no implica que estas dependencias estén instaladas.

6.5 Fallo de GitHub o Git

Compruebe autenticación y remotos:

```
gh auth status
git remote -v
```

Para `commit` y `deploy`, verifique también que la rama admite `push` y que no necesita integrar cambios remotos.

6.6 No hay cambios para desplegar

Compruebe el estado del repositorio:

```
git status
```

Si esperaba cambios en `publish/`, asegúrese de haber ejecutado antes `publish` o utilice `deploy --full`. Construir una publicación no implica por sí solo preparar o desplegar sus artefactos.

6.7 Fallo de Docker

```
iasi-dev docker status  
docker compose version
```

Revise si los puertos configurados ya están ocupados y consulte los logs del servicio afectado con Docker.

6.8 Consultar el log correcto

Busque en el directorio `logs/` del workspace el archivo cuyo nombre corresponde a la operación y a su marca temporal, por ejemplo:

```
iasi-build-YYYYMMDDhhmmss.log  
iasi-publish-YYYYMMDDhhmmss.log  
iasi-pull-YYYYMMDDhhmmss.log
```

Al informar de un problema incluya el comando exacto, la ruta desde la que se ejecutó, el código de salida y el log correspondiente, eliminando previamente credenciales o datos sensibles.